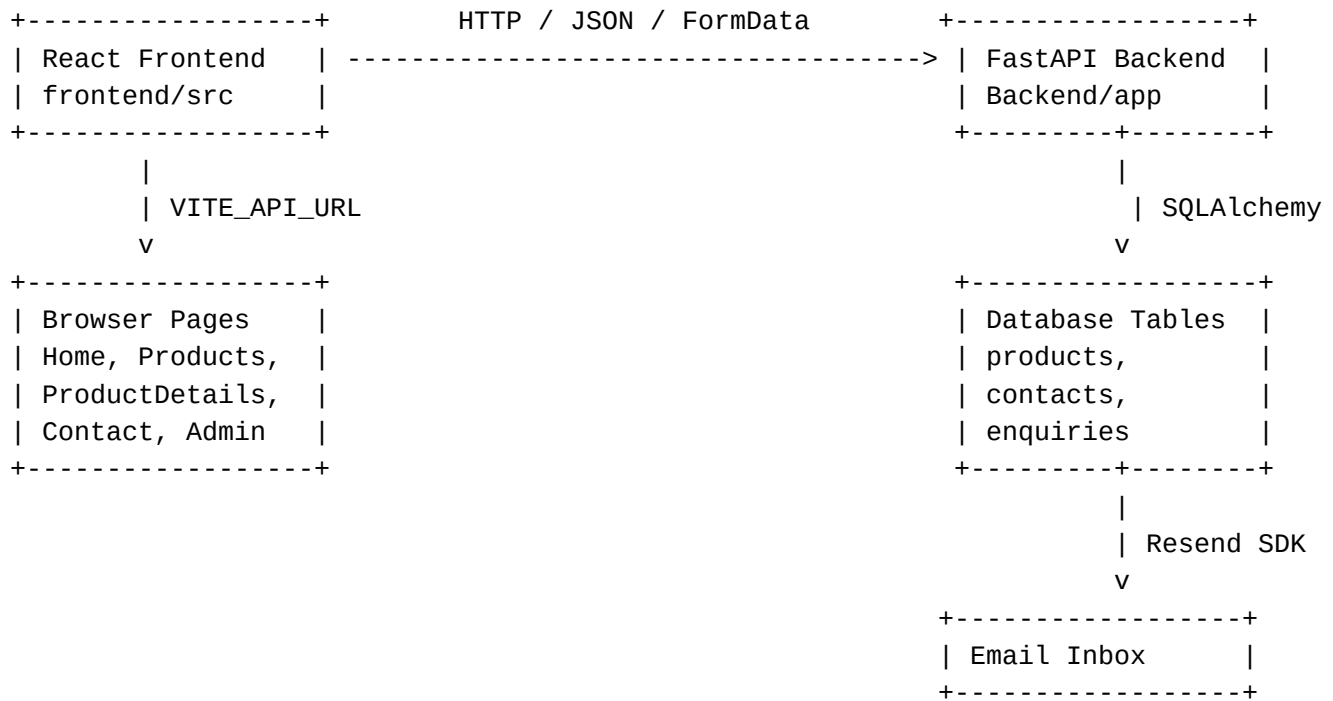


LocalFarm Application Flow

Frontend + Backend Structure and Code Explanation

This PDF explains how the React frontend talks to the FastAPI backend, how products are displayed, how Contact Us and Enquiry forms are saved, and how Resend sends email notifications.

1. High Level Architecture



The frontend is a Vite React app. It reads the backend base URL from VITE_API_URL. The backend is a FastAPI app. It exposes product, contact, and enquiry endpoints, stores records through SQLAlchemy, serves uploaded images from /uploads, and sends email through Resend.

2. Frontend Route Structure

frontend/src/App.jsx

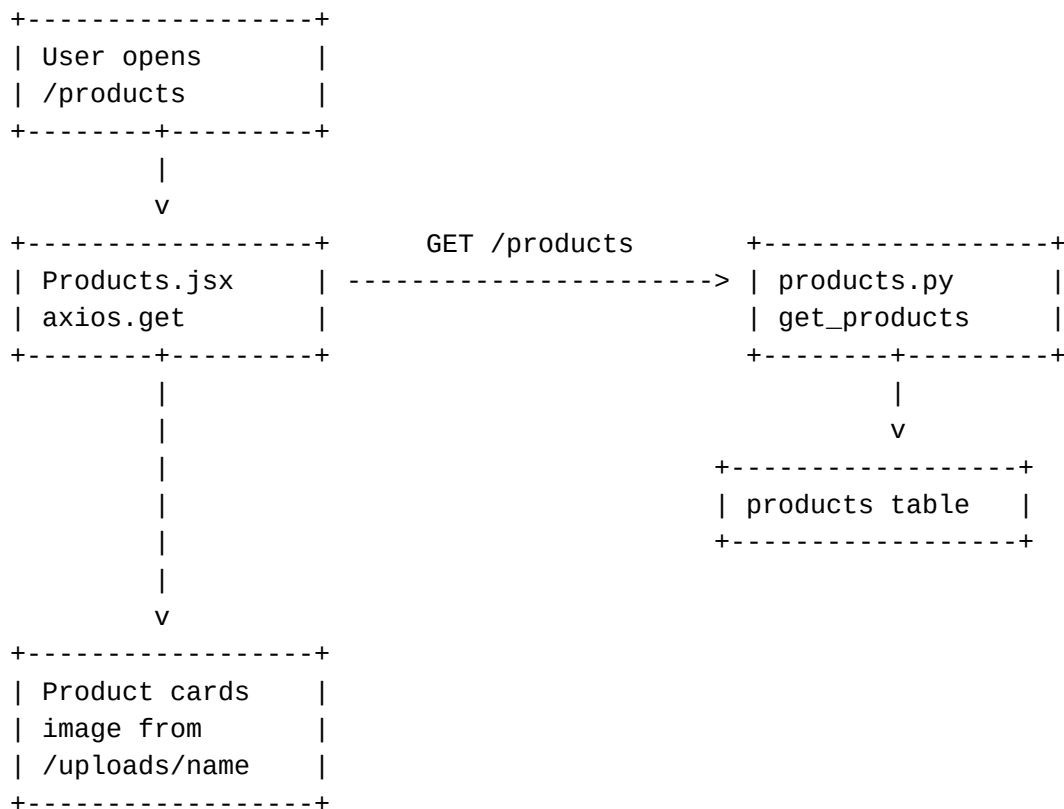
```
/ -> Home.jsx
/products -> Products.jsx
/product/:id -> ProductDetails.jsx
/about -> About.jsx
/contact -> Contact.jsx
/admin -> Login.jsx
/admin/dashboard -> Dashboard.jsx
/admin/add-product -> AddProduct.jsx
```

3. Backend Route Structure

Backend/app/main.py creates FastAPI app and includes routers:

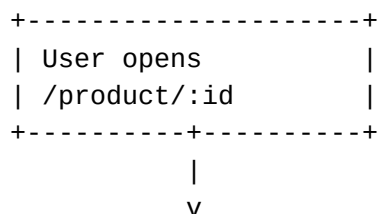
```
GET /products          -> list products
GET /products/{id}     -> get one product
POST /products         -> create product, requires X-API-Key
POST /products/bulk    -> create many products, requires X-API-Key
POST /contact or /contacts -> save contact + send contact mail
POST /enquiry          -> save enquiry + send enquiry mail
GET /uploads/{file}    -> static uploaded product images
```

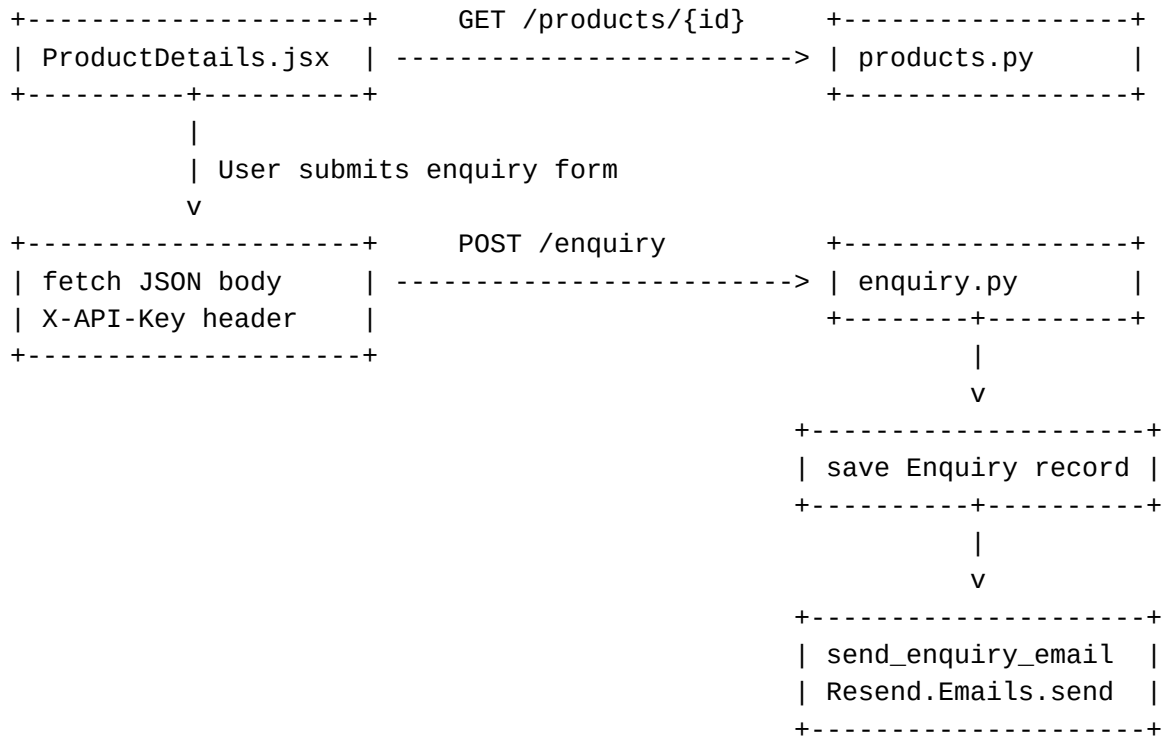
4. Product Listing Flow



Products.jsx loads all products with `axios.get('${API}/products')`. The backend queries the Product table, normalizes image names, and returns a JSON list. The frontend maps that list into product cards. Images are loaded from `'${API}/uploads/<image-name>'`.

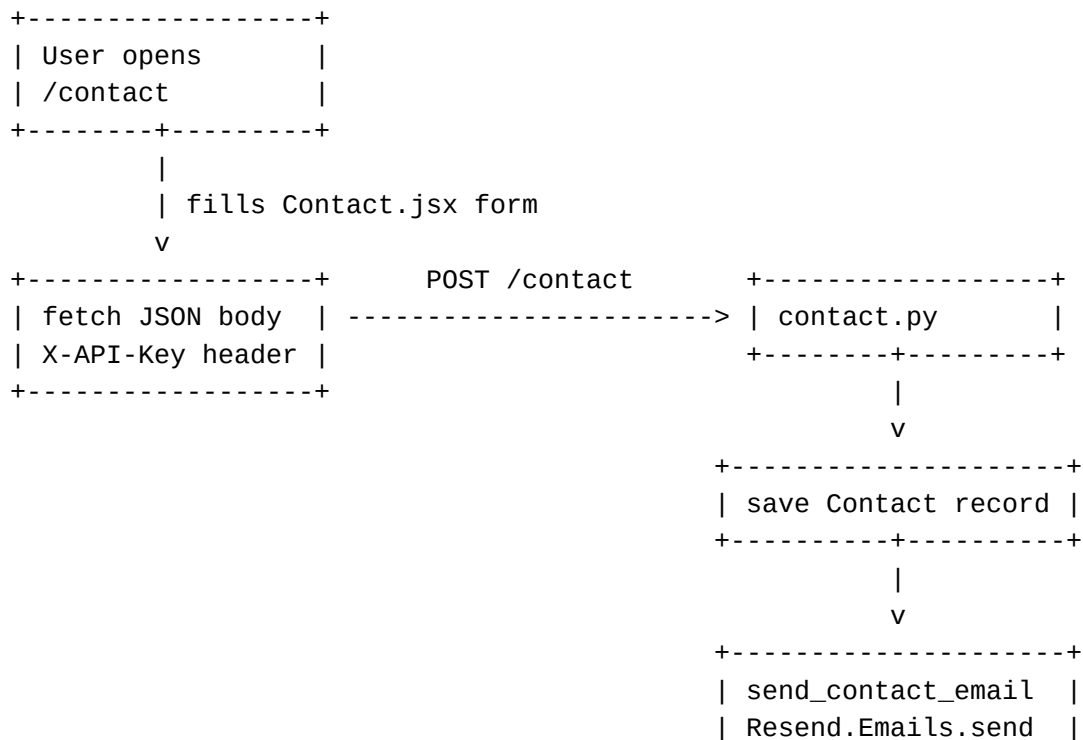
5. Product Detail + Enquiry Flow





ProductDetails.jsx first loads the product using the route id. When the enquiry form is submitted, it validates name, email, phone, and message, then sends JSON to POST /enquiry with X-API-Key. The backend saves the enquiry in the database and calls send_enquiry_email(product_id, name, email, phone, message).

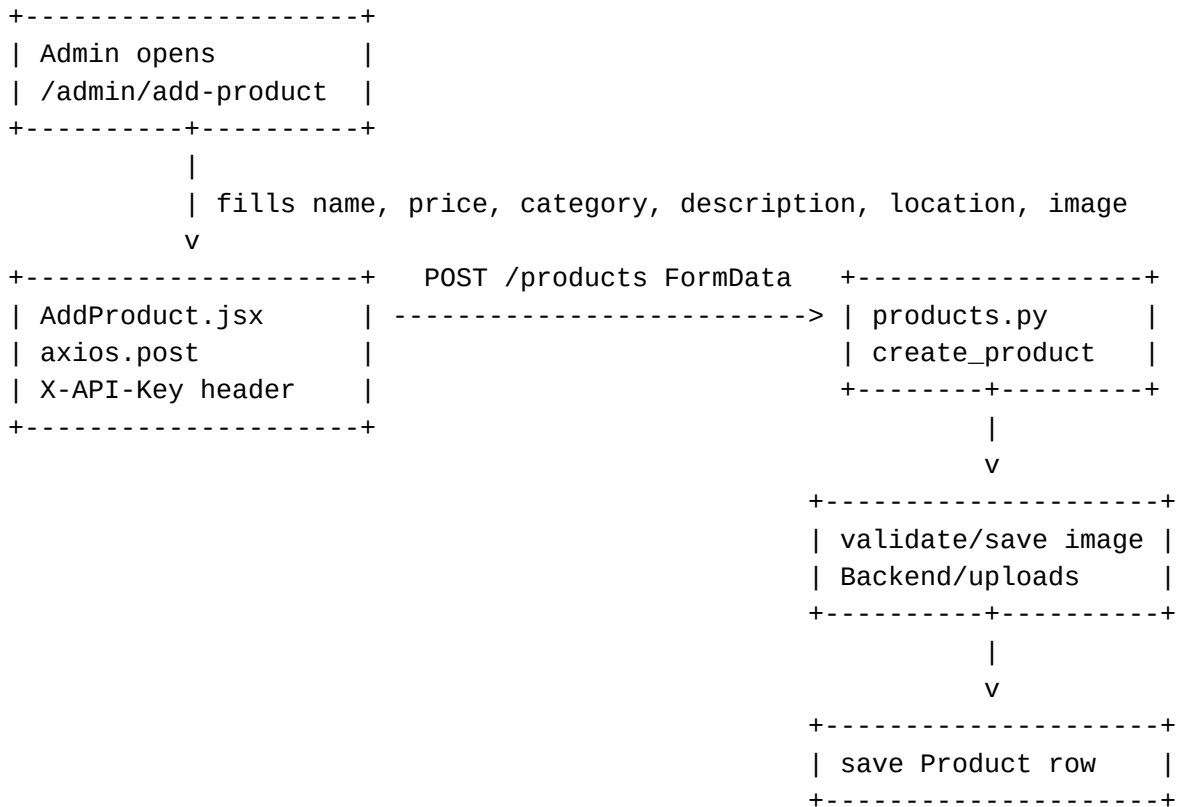
6. Contact Us Flow



+-----+

Contact.jsx validates the contact form and sends name, email, phone, and message to POST /contact. contact.py stores the message in the contacts table. Then email_service.py builds a Resend params dictionary and sends the notification email.

7. Admin Add Product Flow



8. Email Service Code Shape

Backend/app/email_service.py loads:

RESEND_API_KEY -> Resend API key
FROM_EMAIL -> sender, currently onboarding@resend.dev for testing
FROM_NAME -> display name, Local Farm
TO_EMAIL -> receiver email address

The send function builds params like:

```
params = {
    'from': 'Local Farm <onboarding@resend.dev>',
    'to': ['your-email@gmail.com'],
    'subject': 'New Contact Us Message from Customer',
    'html': '<p>...</p>',
    'reply_to': 'customer@example.com'
```

```
}
```

```
email = resend.Emails.send(params)
```

Important Resend rule: `onboarding@resend.dev` is only for testing and can send only to the email address allowed by your Resend account. To send to any customer or other inbox, verify a real domain in Resend and use a sender such as `contact@yourdomain.com`.

9. Backend Database Models

Contact table:

`id, name, email, phone, message`

Enquiry table:

`id, product_id, name, phone, message`

Product table:

`id, name, price, category, description, location, image`

10. Environment Variables

Frontend .env:

`VITE_API_URL` -> backend base URL

`VITE_API_KEY` -> sent as X-API-Key for protected requests

Backend .env:

`DATABASE_URL` -> database connection string

`LOCALFARM_API_KEY` -> accepted backend API key

`RESEND_API_KEY` -> Resend key

`FROM_EMAIL` -> sender address

`FROM_NAME` -> email display name

`EMAIL_ADDRESS` -> receiver address

11. Request Security

`require_api_key()` in `Backend/app/main.py` reads the X-API-Key request header. Protected endpoints use `Depends(require_api_key)`. If the header does not match `LOCALFARM_API_KEY` or fallback key, FastAPI returns HTTP 401 Invalid API key.

12. Quick Code Reading Guide

Start here:

`frontend/src/App.jsx` -> page routes

`frontend/src/pages/Products.jsx` -> product list request

`frontend/src/pages/ProductDetails.jsx` -> detail + enquiry form

`frontend/src/pages/Contact.jsx` -> contact form

`frontend/src/admin/AddProduct.jsx` -> admin product upload

Backend/app/main.py	-> FastAPI setup + API key check
Backend/app/routers/products.py	-> product API
Backend/app/routers/contact.py	-> contact save + mail
Backend/app/routers/enquiry.py	-> enquiry save + mail
Backend/app/email_service.py	-> Resend mail sender
Backend/app/models.py	-> database tables